

Extending the LULC classifier: user control, merging, and sharing

Salil Gujar
M.Tech CSE Entry No. 2025MCS2106

June 2026

Where we are, and what was advised

Done so far

- A 4-class base map over India at 10 m, plus a living hierarchy: split or add a class, give a few example polygons, and retrain that node on the fly.
- A running, git-backed zoo of Model and Dataset cards: browse them, apply a model to the map, and publish to a shared repository, with metrics on each card.

What was advised

- Let users own and share their scheme: choose the base classes, choose the inference data, and save and reload what they built.
- Add a merge operation: relabelling classes across models.
- Recommend where a model applies.
- On publish, store the model, record the contributor, and link the data.

Choosing the starting base classes

- On arrival, the user chooses a starting scheme rather than being fixed to one:
 - IndiaSAT: greenery, water, built-up, barren, the calibrated default.
 - WorldCover (effective): tree, shrubland, grassland, cropland, built-up, bare, water.
- The WorldCover base is trained from WorldCover-labelled Alpha Earth points, keeping only the classes with real coverage over India; the very rare ones are dropped for lack of support.
- It is an alternate starting point, not a more accurate one: WorldCover labels are weak, so it trails IndiaSAT and is presented that way.
- Both bases appear as cards in the zoo, so selecting a base scheme is a first-class action.

Choosing the inference data

- The same trained model can classify a different year. Alpha Earth's embeddings are temporally consistent, so the model is reused and features are simply sampled at the chosen year.
- Realistic mode offers any year from 2017 to 2024. Detailed mode stays at 2024, the only year Tesseract covers over India.
- No retraining is involved, and the chosen year is recorded so a result is reproducible.
- In practice, the same area in 2022 and 2024 shows a genuine difference in the class mix.

Merging classes across models

- A merge is, in essence, a relabelling: take a class produced at one node and a class produced at another, possibly from different models, and declare them one new class.
- It runs as a correction layer applied after the classifiers, not as retraining. This mirrors the idea of pixel-level error-correction layers in the regionally-accurate LULC work.
- For example, relabel tea, from the greenery split, together with mining, from the barren split, into a single extractive class; the map and the counts reflect it at once.
- Merges are stored as rules, so the result is reproducible and a merge can be removed.

Saving and reloading a scheme

- Users combine models in different ways to reach their own classifier. They can now download the hierarchy they built, along with the ordered steps that built it, as a JSON file, and load it back later.
- There are no accounts or sessions: the JSON file is the save. This keeps the tool light while letting people resume exactly where they left off.
- On reload, the tree is validated, each split is rebound to its trained model, and any split that needs retraining is reported.

Recording the order of operations

- The structure of a hierarchy does not, by itself, capture how it was reached. The order of operations matters: a later split or merge depends on what came before.
- Every tree-changing action, split, add, retrain, apply, merge, and base switch, is appended to an ordered operation log.
- The log travels inside the saved scheme, so a shared or restored hierarchy reproduces the same output, and the workflow that produced a unique classifier is preserved.

Recommending where a model applies

- Each model card now suggests where it fits: apply after a given class, derived from the model's node in the hierarchy and any WorldCover mapping its classes carry.
- The suggestion is also aware of the area in view, indicating whether the model is valid for the region currently on screen.
- It is computed from existing card metadata rather than hand-curated, so it stays correct as the zoo grows.

Measuring data spread at a chosen scale

- Training data that is clustered in one area tends to skew a model. We summarise this as a spatial-diversity measure: how evenly the labelled polygons are spread.
- The grid cell on which the spread is measured is now chosen by the user, rather than fixed, so the measure can be read at the scale that matters for a given dataset.
- A finer grid reveals more occupied cells and usually a higher spread; the value updates immediately on the dataset card.

Publishing: storing the model

- When a user publishes, the trained model file is committed into the shared zoo repository. The models are small, so a published model is usable by anyone who pulls it, not merely described.
- The repository continues to hold only cards and these model files; large training tables and tiles are kept out.
- Publishing also records the contributor, a GitHub handle or an email, on the card, so it is clear who shared each model.

Publishing: the data and the contract

- For each training dataset, publishing asks, at that moment, for a public link to where the data came from, and shows it on the card.
- A private upload with no public link is never shared: it stays local and is used only to train the model. The labelled data remains the user's own.
- This keeps the published catalogue rich in provenance while honouring the agreement that the data itself belongs to whoever uploaded it.

Thank you